

SISTEMAS OPERACIONAIS – 2026/II

Contagem Paralela de Objetos em Matriz Binária

Conectividade 8 com Pthreads e Union-Find

Elisa Ely de Oliveira Rigotti | Maria Julia Escobar Correia de Melo

PUCRS – Escola Politecnica | Prof. Filipo Novo Mor

Agenda

- 01 Definição do problema

- 02 Solução sequencial

- 03 Arquitetura paralela

- 04 Validação e testes

- 05 Resultados de desempenho

- 06 Conclusões

PROBLEMA

Contagem de componentes conexos

Dada uma **matriz binária** (0 = fundo, 1 = primeiro plano), contar quantos **objetos** existem usando **conectividade 8**.

Cada objeto é um grupo de células com valor 1 conectadas horizontal, vertical ou diagonalmente.

Requisitos

ANSI C (C89/C90) | Pthreads | Resultados determinísticos



■ Objeto 1 ■ Objeto 2 ■ Objeto 3

REFERÊNCIA

Versão sequencial - BFS iterativa

Algoritmo

Percorre a matriz célula a célula. Ao encontrar um 1 não visitado, inicia uma BFS iterativa marcando todo o componente.

Por que BFS iterativa?

Evita stack overflow em componentes grandes. Usa fila explícita ao invés de recursão.

Pseudocódigo

```
para cada célula (i, j):  
  se matriz[i][j] == 1  
  e não visitado:  
    BFS(i, j)  
    objetos++
```

Complexidade: $O(L \times C)$
8 vizinhos por célula

PARALELISMO

Arquitetura paralela

Modelo

Pthreads com decomposição por faixas de linhas (row-stripe)

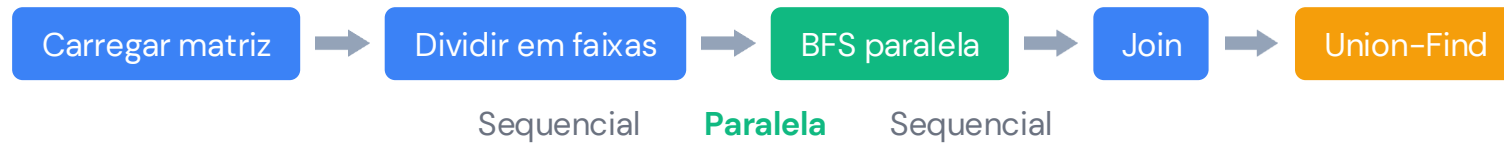
Trabalhadores

Configurável via argumento de linha de comando

Sincronizacao

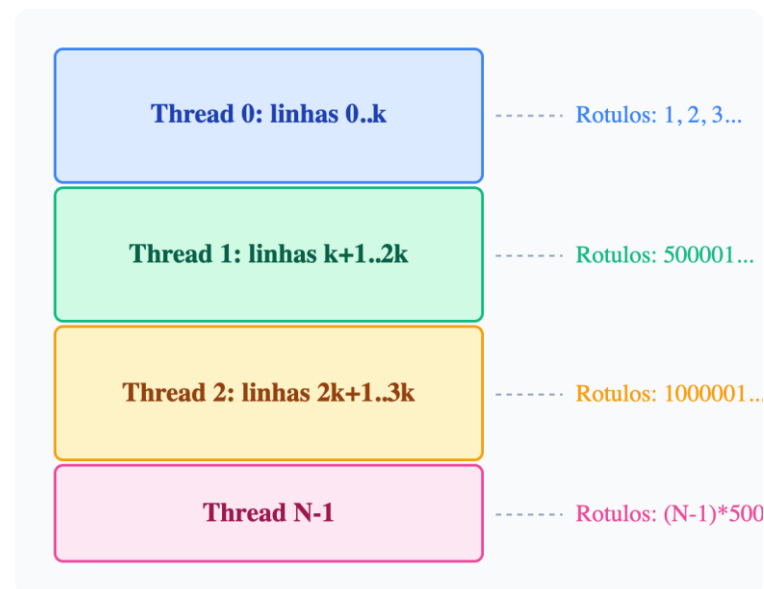
pthread_join como barreira implícita.
Nenhum mutex necessário.

Fluxo de execucao



DECOMPOSIÇÃO

Divisão por faixas de linhas



Divisão estática

Cada thread recebe L/N linhas. As primeiras $L\%N$ threads recebem uma linha extra.

Rótulos únicos

$\text{base_rotulo} = t * 500000$. Rótulos de threads diferentes nunca colidem.

Sem sincronização

Faixas disjuntas garantem ausência de conflito na escrita.

CONSOLIDAÇÃO

Union-Find nas fronteiras

Apos a BFS paralela, objetos que cruzam fronteiras entre faixas possuem **rótulos diferentes**. A consolidação percorre cada fronteira e unifica rótulos equivalentes.

Direções verificadas na fronteira

S (abaixo)

SW (diagonal)

SE (diagonal)

Otimizacoes do Union-Find

- ✓ Compressão de caminho (path splitting)
- ✓ União por rank
- ✓ Executado apos pthread_join (sem locks)

Exemplo rastreável

rotulo[5][5] = 1 (Thread 0)

rotulo[6][6] = 500001 (Thread 1)

uf_union(1, 500001)

Resultado: mesmo componente

Sincronização e correção

Sem condições de corrida

Threads escrevem em faixas disjuntas da matriz de rótulos. Sem sobreposição = sem conflito.

Sem deadlocks

Nenhum mutex ou lock no programa. Única sincronização: barreira implícita do pthread_join.

Determinístico

Resultados idênticos entre sequencial e paralelo em todas as configurações testadas.

Recurso	Risco	Protecao	Justificativa
matriz[][]	Nenhum	Nenhuma	Somente leitura
rotulo[][]	Escrita concorrente	Particionamento	Faixas disjuntas
Union-Find	Corrida potencial	Fase sequencial	Apos pthread_join
Fila BFS	Nenhum	Alocacao local	Privada por thread

TESTES

Validação funcional

5 matrizes obrigatórias – Todas aprovadas

Exemplo	Dimensões	Esperado	Seq	Par
1	5 x 5	3	3	3
2	6 x 8	4	4	4
3	8 x 8	5	5	5
4	9 x 12	6	6	6
5	12 x 12	7	7	7

Matrizes adicionais

Dimensoes	Objetos	Threads	Status
100 x 100	461	1-8	OK
500 x 500	11.947	1-8	OK
1000 x 1000	47.911	1-8	OK
2000 x 2000	189.564	1-8	OK

5 repetições por configuração. Resultado sempre idêntico entre versões.

Metodologia experimental

Ambiente

Apple M4 (4P + 6E)
10 núcleos | 16 GB RAM
macOS Sequoia 15.7.3 | arm64

Medição

clock_gettime(CLOCK_MONOTONIC)
Trecho: threads + BFS + join + consolidação
Exclui leitura do arquivo

Matrizes testadas

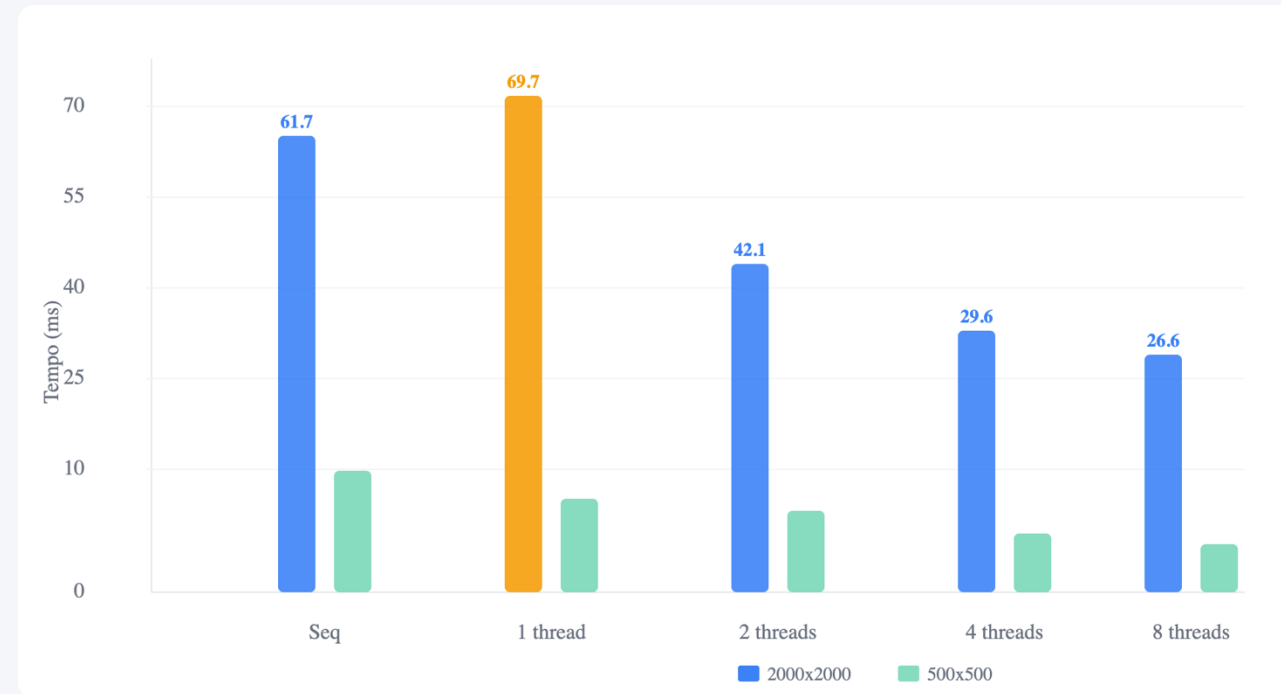
100x100 | 500x500 | 1000x1000 | 2000x2000
Densidade: 30% | Semente: 42

Configuração

Threads: 1, 2, 4, 8
5 repetições por configuração
Medida: média aritmética

DESEMPENHO

Tempo de execução



2000x2000

61.7 ms

sequencial

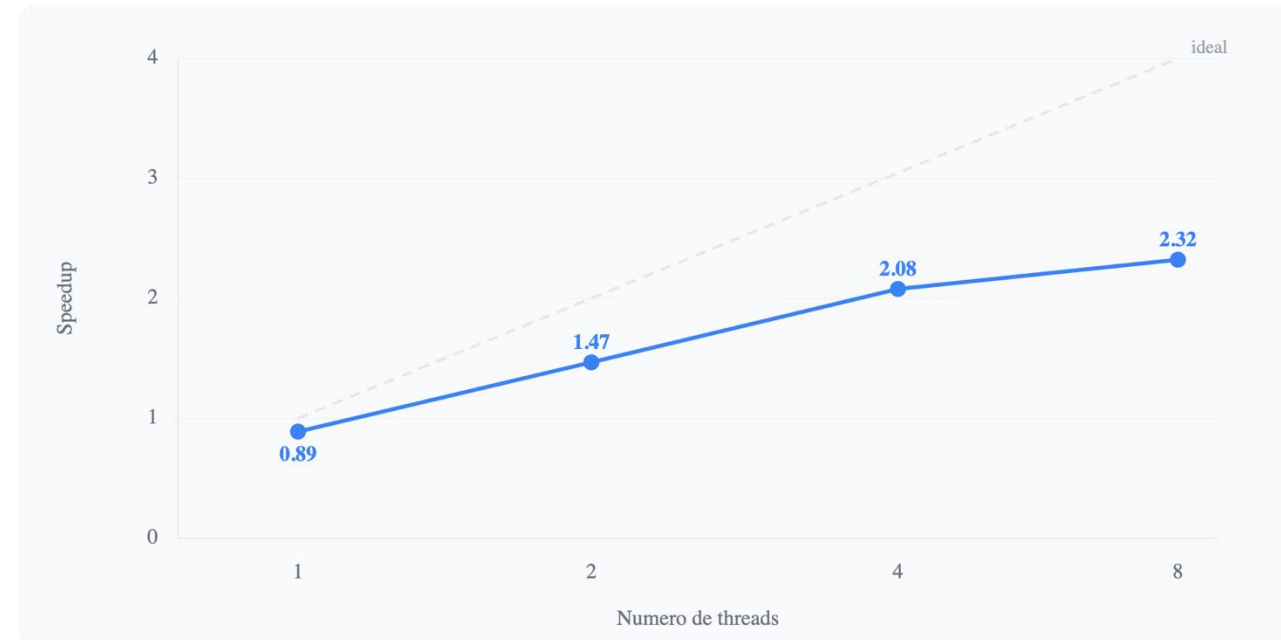
8 threads

26.6 ms

-57% tempo

ACELERAÇÃO

$$\text{Speedup } S(p) = T_{\text{seq}} / T_{\text{par}}(p)$$



Melhor speedup

2.32x

8 threads, 2000x2000

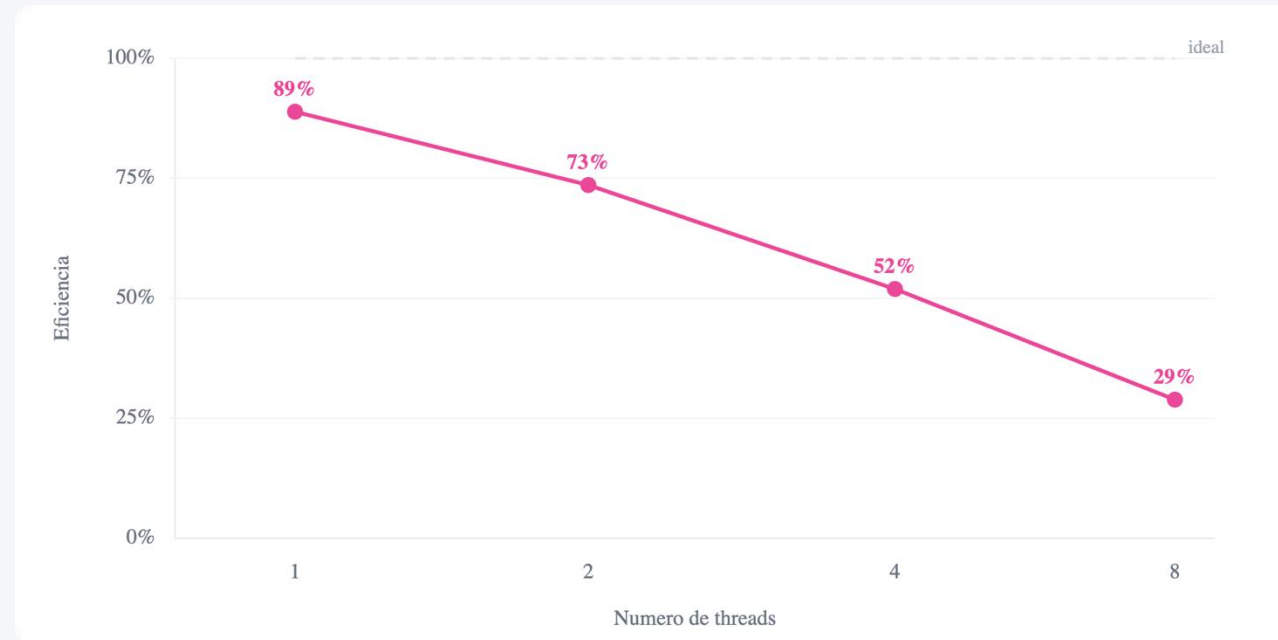
1 thread

0.89x

overhead de threads e Union-Find

EFICIÊNCIA

$$\text{Eficiência } E(p) = S(p) / p$$



Melhor eficiência

89%

1 thread (overhead mínimo)

8 threads

29%

Custos de gestão dominam

Eficiência decresce com mais threads: overhead de criação e consolidação cresce mais rápido que o ganho.

Análise de desempenho

Overhead com 1 thread

Speedup 0.89x indica que a infraestrutura de threads (pthread_create, Union-Find, consolidacao) custa ~11% sobre o sequencial puro.

Efeito do tamanho

Matrizes maiores se beneficiam mais do paralelismo. Em 500x500, os tempos são tão curtos que o overhead domina. O ganho real aparece a partir de 1000x1000.

Lei de Amdahl

A fase sequencial (Union-Find + consolidacao) limita o speedup maximo teórico. Com fração serial estimada em ~15%, o limite seria ~6.7x.

Arquitetura do M4

Apple M4 tem 4 núcleos P + 6 núcleos E. Com 8 threads, os núcleos E (mais lentos) participam, reduzindo a eficiência por thread.

2-4

threads ideais

52-73%

faixa de eficiência

~15%

fração serial estimada

CONCLUSOES

Resultados e aprendizados

Desempenho

Speedup de 2.32x com 8 threads na matriz 2000x2000. Reducao de 57% no tempo de execucao (61.7ms para 26.6ms).

Correcao

100% dos testes aprovados. Resultados idênticos entre sequencial e paralelo em todas as configurações. Sem race conditions ou deadlocks.

Aprendizados

Programação concorrente com Pthreads, estratégias de decomposição de dados e estrutura Union-Find para consolidação.

Trabalho futuro

Explorar balanceamento dinâmico de carga, Union-Find lock-free com operações atômicas, e suporte a matrizes esparsas para melhorar escalabilidade.

2.32x
melhor speedup

Perguntas?

Obrigada pela atenção! Estamos a disposição para perguntas e discussão.

Elisa Rigotti & Maria Julia Escobar Correia de Melo

Sistemas Operacionais – PUCRS 2026/2